Détection d'Anomalies Bancaires

à l'aide de Solveurs Itératifs et Régression Logistique

Architecture d'un système de détection de fraudes bancaires : de l'algèbre matricielle creuse aux solveurs itératifs, de la régression logistique à partir de zéro jusqu'à l'explicabilité du modèle par valeurs de Shapley.

Auteur : Mattéo Lebarbier

Mars 2026

0. Table des matières

1 Introduction **3**

2 Architecture du projet **4**

3 Données transactionnelles simulées **4**
3.1 Génération de la matrice 4
3.2 Le format CSR 5

4 Formulation algébrique **5**
4.1 Des équations normales au système linéaire 5
4.2 Propriétés spectrales de la matrice A 6

5 Solveurs itératifs **6**
5.1 Principe général et critère d'arrêt 6
5.2 Méthode de Jacobi 6
5.3 Méthode de Gauss-Seidel 7
5.4 Successive Over-Relaxation (SOR) 7
5.5 Résultats comparatifs 7

6 Régression Logistique **8**
6.1 Du score linéaire à la probabilité 8
6.2 Fonction de coût et gradient 8
6.3 Descente de gradient mini-batch 8
6.4 Gestion du déséquilibre - Class Weighting 9

7 SMOTE - Équilibrage des classes **9**
7.1 Principe algorithmique 9
7.2 Analyse comparative 10

8 Évaluation et résultats **10**
8.1 Protocole 10
8.2 Métriques finales 10
8.3 Matrice de confusion - sans SMOTE (jeu de test, 1 000 transactions) 11

9 Analyse d'explicabilité SHAP **11**
9.1 Cadre théorique 11
9.2 Importance globale des features 11
9.3 Explication individuelle - Waterfall plot 12

9.4 Limites 13

10 Pipeline d'inférence **13**

11 Conclusion **13**

Glossaire des concepts clés **15**

1. Introduction

Ce rapport présente la conception et l'implémentation complète d'un pipeline de détection d'anomalies bancaires, construit entièrement à partir des fondements de l'algèbre linéaire. L'objectif central est double : démontrer que les outils de l'analyse matricielle constituent le socle commun du machine learning et de l'optimisation numérique, et produire un système aussi performant qu'auditable, capable d'expliquer chacune de ses décisions.

La détection de fraude est un problème de classification binaire à fort déséquilibre : dans un flux transactionnel réaliste, moins de 5 % des opérations sont frauduleuses. Cette asymétrie constitue le premier défi technique : un classifieur naïf qui prédit systématiquement *légitime* atteint 95 % d'accuracy sans jamais détecter une seule fraude. Résoudre ce problème sans recourir à des bibliothèques de haut niveau exige une maîtrise précise des mécanismes sous-jacents.

La contribution principale de ce travail est de montrer que le gradient de la *log-loss*, les équations normales de la régression Ridge, et le critère de convergence des solveurs itératifs se ramènent tous à la même opération fondamentale : le produit matrice-vecteur creux $X^\top v$. Cette unification algébrique n'est pas un artifice pédagogique : elle a des conséquences pratiques directes sur la conception du pipeline et l'exploitation des structures de données creuses.

Le projet s'articule autour de quatre axes techniques :

- **Structures de données creuses (format CSR)** : exploitation de la sparsité naturelle des transactions pour réduire l'empreinte mémoire d'un facteur 100.
- **Solveurs itératifs** : implémentation *from scratch* de Jacobi, Gauss-Seidel et SOR, dont le comportement est analysé au regard des propriétés spectrales de la matrice du système.
- **Régression logistique** : entraînement par descente de gradient mini-batch avec *class weighting* et seuil optimal, sans recourir à `sklearn.fit()`.
- **Explicabilité SHAP** : attribution des contributions individuelles de chaque feature à la décision de détection, dans le cadre de la théorie des jeux coopératifs.

Résultat principal. ROC-AUC de 0,967 sur le jeu de test (sans SMOTE). Avec SMOTE : rappel de 74 %, soit 37 fraudes détectées sur 50. L'ensemble du pipeline est implémenté *from scratch* en Python.

2. Architecture du projet

Le pipeline est organisé en sept modules Python à responsabilités séparées. Cette modularité garantit la testabilité unitaire de chaque composant et facilite le remplacement indépendant de l'une des briques (par exemple, substituer le solveur GS par une factorisation de Cholesky sans modifier le module de régression logistique).

Table 1 – Modules Python du pipeline

Module	Responsabilité
`data_simulation.py`	Génération des transactions (matrice CSR, labels, SMOTE)
`solvers.py`	Solveurs itératifs : `build_system()`, `jacobi()`, `gauss_seidel()`, `sor()`
`logistic_regression.py`	Entraînement, optimisation du seuil τ^*, évaluation
`predict.py`	Sérialisation du modèle (`.npz`), scoring en inférence
`plots.py`	Courbes de convergence, ROC, matrices de confusion
`shap_analysis.py`	Importance globale, waterfall plot, scatter SHAP
`main.py`	Orchestration séquentielle des six modules

3. Données transactionnelles simulées

3.1 Génération de la matrice

Les données bancaires réelles étant soumises à des contraintes de confidentialité strictes, la matrice $X \in \mathbb{R}^{5000\times 200}$ est simulée de façon à reproduire les propriétés structurelles d'un flux transactionnel réel : forte sparsité, déséquilibre de classes 95/5, et features continues positives bornées.

```
X = sparse.random(5000, 200, density=0.01, format='csr',
    random_state=42)
true_weights = np.random.rand(200)
scores = X.dot(true_weights)
y = (scores > np.percentile(scores, 95)).astype(float)
```

Listing 1 – Génération de la matrice creuse et des labels

La matrice présente une densité de 1 %, soit environ 10 000 valeurs non nulles sur un million d'entrées théoriques. Les labels sont construits via un vrai vecteur de poids $w^* \in \mathbb{R}^{200}$ et

un seuil au 95e percentile, garantissant exactement 5 % de fraudes - taux réaliste pour ce type de problème.

3.2 Le format CSR

Le format *Compressed Sparse Row* représente une matrice creuse M via trois tableaux dont la taille est proportionnelle au nombre de valeurs non nulles nnz, indépendamment des dimensions de la matrice :

- `data` $\in \mathbb{R}^{\text{nnz}}$ - valeurs non nulles, parcourues ligne par ligne ;
- `indices` $\in \mathbb{N}^{\text{nnz}}$ - indice de colonne de chaque valeur ;
- `indptr` $\in \mathbb{N}^{n+1}$ - tel que `data`[`indptr`[i]:`indptr`[i+1]] contient les valeurs de la ligne i.

Table 2 – Comparaison mémoire : format dense vs. CSR

Format	Éléments stockés	Mémoire (float64)
Dense	1 000 000	≈ 8 Mo
CSR	∼10 000 + overhead	≈ 80 Ko
Facteur	**100×**	**100×**

4. Formulation algébrique

4.1 Des équations normales au système linéaire

On cherche $w \in \mathbb{R}^p$ minimisant la fonctionnelle de coût Ridge :

$$\mathcal{L}(w) = \|Xw - y\|_2^2 + \lambda \|w\|_2^2$$

En annulant le gradient $\nabla_w \mathcal{L} = 2X^\top(Xw - y) + 2\lambda w = 0$, on obtient les équations normales régularisées :

$$\underbrace{(X^\top X + \lambda I)}_{A}\, w = \underbrace{X^\top y}_{b} \quad \Longleftrightarrow \quad Aw = b$$

Ce résultat est l'insight central du projet : un problème d'optimisation ML se transforme en un système linéaire carré $Aw = b$ de dimension $p \times p = 200 \times 200$, résolvable par des méthodes itératives classiques de l'analyse numérique. La régularisation Ridge n'est pas seulement un outil de régularisation statistique : elle conditionne les propriétés spectrales de A et donc la convergence des solveurs.

4.2 Propriétés spectrales de la matrice A

La matrice $A = X^\top X + \lambda I$ est **symétrique définie positive** (SDP) par construction :

— *Symétrie* : $A^\top = (X^\top X)^\top + \lambda I = X^\top X + \lambda I = A$
— *Définition positive* : $\forall v \neq 0,\ v^\top A v = \|Xv\|^2 + \lambda \|v\|^2 > 0$ (car $\lambda > 0$)

Table 3 – Propriétés spectrales de la matrice A

Propriété	Valeur / Implication
Symétrique définie positive	Convergence GS garantie (théorème d'Ostrowski-Reich)
Nombre de conditionnement	$\kappa \approx 7$ - système bien conditionné, convergence rapide
Dominance diagonale stricte	1,5 % des lignes seulement - Jacobi diverge
Valeur propre maximale	$\lambda_{\max} \approx 42{,}9$
Valeur propre minimale	$\lambda_{\min} \approx 6{,}2$ (relevée par la régularisation Ridge)

Le faible conditionnement $\kappa \approx 7$ est une conséquence directe de la régularisation : le terme λI relève toutes les valeurs propres d'au moins λ, réduisant le ratio $\lambda_{\max}/\lambda_{\min}$. Cette propriété est fondamentale pour comprendre pourquoi SOR n'apporte aucun gain ici.

5. Solveurs itératifs

5.1 Principe général et critère d'arrêt

Tous les solveurs partent de $w^{(0)} = 0$ et produisent une suite $\{w^{(k)}\}$ jusqu'à satisfaire le critère :

$$\|Aw^{(k)} - b\|_2 < \text{tol} = 10^{-6}$$

Ce seuil est un critère d'arrêt choisi, pas une contrainte physique du système. Pour $\kappa \approx 7$, descendre à 10^{-9} serait inutilement coûteux sans gain mesurable sur les prédictions finales.

5.2 Méthode de Jacobi

À chaque itération k, toutes les composantes sont mises à jour *simultanément* en utilisant uniquement $w^{(k)}$:

$$w_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}\, w_j^{(k)} \right)$$

La convergence est régie par le rayon spectral $\rho(M_J)$ de la matrice d'itération de Jacobi $M_J = -D^{-1}(L + U)$. La condition nécessaire et suffisante est $\rho(M_J) < 1$, ce qui requiert

en pratique la dominance diagonale stricte. Avec seulement 1,5 % des lignes dominantes, Jacobi diverge exactement comme prédit par la théorie.

5.3 Méthode de Gauss-Seidel

La modification est minimale mais décisive : $w_i^{(k+1)}$ est utilisé immédiatement pour les composantes suivantes dans la même itération.

```
for i in range(n):
    s = np.dot(A[i], w) - A[i, i] * w[i]
    w[i] = (b[i] - s) / A[i, i]  # mise a jour immediate, pas de w_new
```

Listing 2 – Implémentation de Gauss-Seidel

Pour une matrice SDP, le **théorème d'Ostrowski-Reich** garantit la convergence inconditionnelle de Gauss-Seidel, quelle que soit l'initialisation. Résultat : 14 itérations contre divergence complète pour Jacobi. L'exploitation des valeurs déjà calculées dans l'itération courante accélère la propagation de l'information dans le vecteur solution.

5.4 Successive Over-Relaxation (SOR)

SOR généralise Gauss-Seidel par un paramètre de relaxation $\omega \in (0, 2)$:

$$w_i^{(k+1)} = \omega \cdot w_i^{\mathrm{GS}} + (1 - \omega) \cdot w_i^{(k)}$$

Avec $\omega = 1{,}2$, SOR converge en 21 itérations, plus lentement que Gauss-Seidel. Ce résultat est cohérent avec la théorie : la sur-relaxation accélère la convergence uniquement lorsque κ est grand (système mal conditionné). Pour $\kappa \approx 7$, Gauss-Seidel est déjà proche de l'optimum et la sur-relaxation introduit de l'instabilité sans bénéfice.

5.5 Résultats comparatifs

Table 4 – Comparaison des solveurs itératifs

Solveur	Itérations	Résidu final	Statut
Jacobi	500	$4{,}16 \times 10^{88}$	Diverge
Gauss-Seidel	14	$8{,}59 \times 10^{-7}$	Converge
SOR ($\omega = 1{,}2$)	21	$\approx 10^{-7}$	Converge

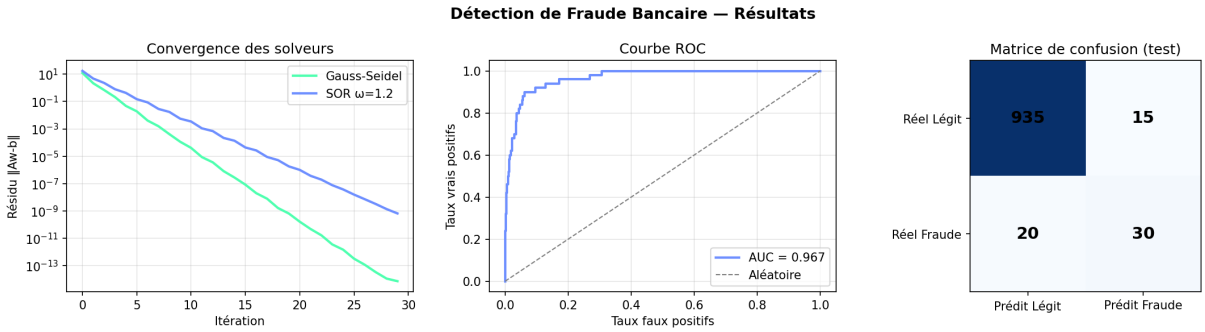

Figure 1 – Convergence des solveurs (gauche), courbe ROC avec AUC = 0,967 (centre) et matrice de confusion sur le jeu de test (droite).

6. Régression Logistique

6.1 Du score linéaire à la probabilité

Le vecteur w issu des solveurs produit un score non borné $s_i = x_i^\top w$. La régression logistique transforme ce score en probabilité de fraude via la sigmoïde :

$$P(\text{fraude} = 1 \mid x_i) = \sigma(x_i^\top w + b) = \frac{1}{1 + e^{-(x_i^\top w + b)}}$$

où $b \in \mathbb{R}$ est un biais appris séparément.

6.2 Fonction de coût et gradient

La log-loss régularisée Ridge s'écrit :

$$\mathcal{L}(w) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \Big] + \frac{\lambda}{2} \|w\|_2^2$$

Son gradient par rapport à w est :

$$\nabla_w \mathcal{L} = \frac{1}{n_b} X_b^\top \Big(s \odot (\hat{y} - y) \Big) + \lambda w$$

où s est le vecteur de pondération et $\odot$ le produit de Hadamard. Ce gradient est un **produit matrice-vecteur creux** - exactement la même opération que dans les solveurs itératifs. C'est le fil conducteur algébrique de tout le projet.

6.3 Descente de gradient mini-batch

```python
for epoch in range(45):              # 45 epoques
    indices = np.random.permutation(n)  # shuffle
    for start in range(0, n, 256): # batchs de 256
        y_hat = sigmoid(X_b @ w + b)
        err   = sample_weights * (y_hat - y_b)
        grad_w = X_b.T.dot(err) / n_b + lambda_r * w
        w -= lr * grad_w
        b -= lr * err.mean()
```

Listing 3 – Boucle d'entraînement mini-batch

Paramètres : lr $= 0{,}1$, $\lambda = 10^{-3}$, batch_size $= 256$, 45 époques. La loss passe de 0,690 à 0,574 sur le train.

6.4 Gestion du déséquilibre - Class Weighting

Avec 5 % de fraudes, la log-loss non pondérée est dominée par les transactions légitimes, conduisant à la solution dégénérée « tout légitime ». La pondération corrige cela en amplifiant la contribution des fraudes dans le gradient :

$$s_i = \frac{n}{2 \cdot n_{\text{fraude}}} \approx 10 \quad \text{si } y_i = 1 \qquad s_i = \frac{n}{2 \cdot n_{\text{légit}}} \approx 0{,}53 \quad \text{si } y_i = 0$$

Le seuil de décision optimal τ^* est sélectionné en maximisant le F1-score sur le jeu d'entraînement ($\tau^* = 0{,}53$). Il n'est jamais utilisé pour la sélection sur le jeu de test, évitant ainsi tout *data leakage*.

7. SMOTE - Équilibrage des classes

7.1 Principe algorithmique

SMOTE (*Synthetic Minority Oversampling Technique*) génère des exemples synthétiques de la classe minoritaire par interpolation linéaire dans l'espace des features :

$$\tilde{x} = x_i + \delta \cdot (x_{\text{knn}} - x_i), \quad \delta \sim \mathcal{U}(0,\, 1)$$

```python
from imblearn.over_sampling import SMOTE
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train.toarray(), y_train)
# Avant : 3800 legitimes, 200 fraudes  (ratio 95/5)
```

```
# Apres  : 3800 legitimes, 3800 fraudes (ratio 50/50)
```

Listing 4 – Application de SMOTE avec imblearn

7.2 Analyse comparative

Table 5 – Comparaison class weighting vs. SMOTE

Métrique	Sans SMOTE	Avec SMOTE	Variation
Rappel	0,600	0,740	+23 %
Précision	0,667	0,416	−38 %
ROC-AUC	0,967	0,958	−0,9 %
Fausses alarmes/1000	15	52	+247 %

Le choix entre class weighting et SMOTE est un **arbitrage métier** : SMOTE maximise le rappel au prix d'un taux de fausses alarmes 3,5× plus élevé. En environnement bancaire, 52 blocages injustifiés par millier de transactions représentent un coût opérationnel significatif. Le class weighting constitue un point de départ plus conservateur.

8. Évaluation et résultats

8.1 Protocole

Le jeu de données est divisé en 80 % train / 20 % test avec stratification (paramètre `stratify=y`) pour préserver le ratio 95/5 dans les deux splits. Le seuil τ^* est optimisé uniquement sur le train et appliqué sans modification sur le test.

8.2 Métriques finales

Table 6 – Métriques de performance - sans et avec SMOTE

Métrique	Sans SMOTE	Avec SMOTE	Définition
Rappel (Sensibilité)	0,600	0,740	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$
Précision (VPP)	0,667	0,416	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$
F1-score	0,632	0,537	$2 \cdot P \cdot R/(P+R)$
Spécificité	0,984	0,945	$\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$
ROC-AUC	0,967	0,958	Indép. du seuil

8.3 Matrice de confusion - sans SMOTE (jeu de test, 1 000 transactions)

Table 7 – Matrice de confusion sur le jeu de test (sans SMOTE)

		Prédit	
		Légitime	Fraude
Réel	Légitime (950)	TN = 935	FP = 15
	Fraude (50)	FN = 20	TP = 30

9. Analyse d'explicabilité SHAP

9.1 Cadre théorique

L'explicabilité est une exigence réglementaire en détection de fraude bancaire (RGPD, directive PSD2) : toute décision automatisée impactant un client doit pouvoir être justifiée. SHAP (*SHapley Additive exPlanations*) fournit cette justification en s'appuyant sur la théorie des jeux coopératifs de Shapley (1953).

La valeur de Shapley ϕ_i d'une feature i est définie comme sa contribution marginale moyenne, pondérée sur toutes les coalitions possibles de features :

$$\phi_i(f, x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \Big[f(S \cup \{i\}) - f(S) \Big]$$

Pour un modèle linéaire $f(x) = x^\top w + b$, le calcul est exact et fermé : $\phi_i = w_i \cdot (x_i - \mathbb{E}[x_i])$. Le calcul est donc instantané via `shap.LinearExplainer`, sans approximation Monte Carlo.

9.2 Importance globale des features

Table 8 – Top 5 features par importance SHAP moyenne $|\mathbb{E}[\phi_i]|$

Rang	**Feature**	**SHAP moyen** $|\mathbb{E}[\phi_i]|$
1	Feature 25	0,0159
2	Feature 48	0,0154
3	Feature 74	0,0098
4	Feature 6	0,0087
5	Feature 160	0,0082

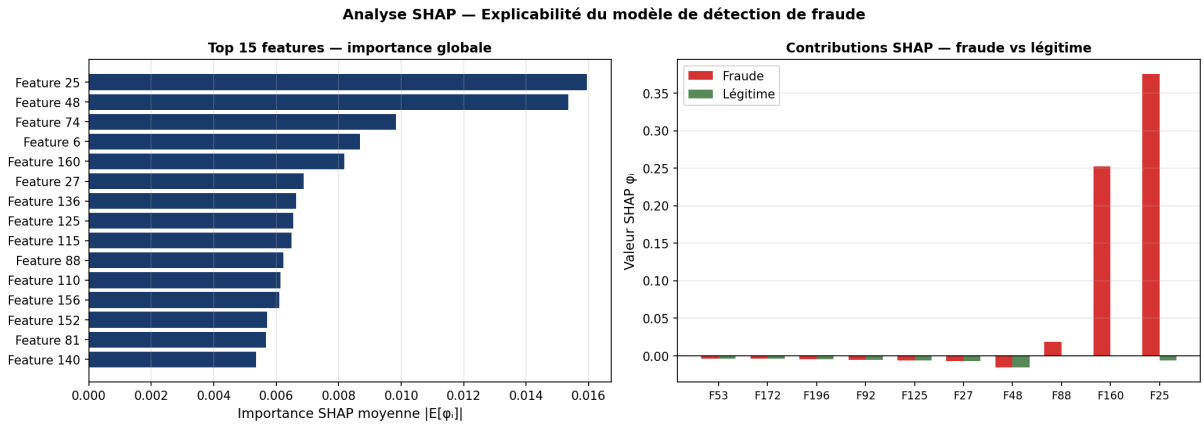

Figure 2 – Top 15 features par importance SHAP moyenne (gauche) et contributions SHAP fraude vs. légitime pour une sélection de features (droite). Les features 25 et 48 dominent, cohérentes avec les poids w appris.

La concentration de l'importance sur un petit nombre de features est cohérente avec la sparsité des données : chaque transaction n'activant que $\sim$2 features en moyenne, seules quelques features apparaissent fréquemment dans les transactions frauduleuses et acquièrent un poids élevé.

9.3 Explication individuelle - Waterfall plot

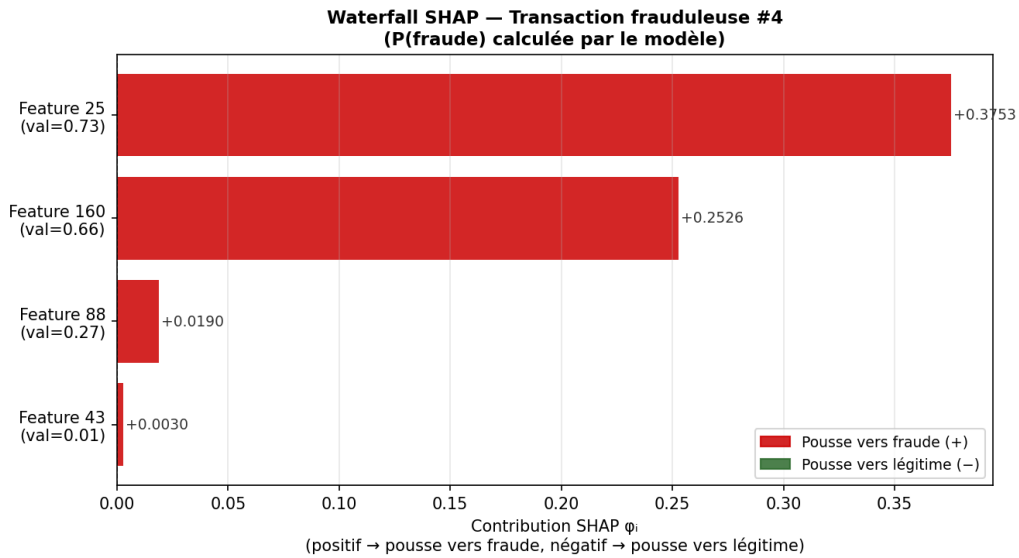

Figure 3 – Waterfall SHAP pour la transaction frauduleuse #4. La Feature 25 (val = 0,73) contribue +0,375 au log-odds de fraude, la Feature 160 (val = 0,66) contribue +0,253.

Ce graphique répond à la question opérationnelle : *pourquoi cette transaction précise a-t-elle été flaggée ?* La Feature 25, avec une valeur observée de 0,73, augmente le log-odds de fraude de +0,375, interprétable directement comme la contribution causale (au sens de Shapley) de cette feature à la décision. Avec des features nommées (montant, heure,

pays, catégorie marchant), cette décomposition est directement exploitable par les équipes antifraude.

9.4 Limites

Pour un modèle linéaire, les valeurs SHAP sont formellement redondantes avec les poids w : $\phi_i \propto w_i \cdot x_i$. La véritable valeur ajoutée de SHAP apparaît sur des modèles non linéaires (XGBoost, réseaux de neurones) où les interactions entre features rendent les poids non interprétables directement. L'intégration de SHAP dans ce projet est une démonstration de la chaîne *explicabilité-as-code* plutôt qu'une nécessité analytique.

10. Pipeline d'inférence

Le modèle entraîné est sérialisé au format `.npz` (NumPy natif), évitant toute dépendance à `pickle` ou à des formats propriétaires. Le scoring d'une nouvelle transaction s'effectue en $\mathcal{O}(p)$ opérations :

```
from predict import load_model, score_transaction

w, b, threshold = load_model('fraud_model.npz')
tx = np.zeros(200)
tx[3], tx[17], tx[42] = 0.8, 0.5, 1.2
proba, verdict = score_transaction(tx, w, b, threshold)
# P(fraude) = 0.5814 | Verdict = FRAUDE
```

Listing 5 – Scoring d'une nouvelle transaction

Table 9 – Niveaux d'alerte et actions opérationnelles

Niveau	Seuil	Action opérationnelle
MINIMAL	$P < 0{,}2$	Transaction autorisée
LOW	$0{,}2 \leq P < 0{,}5$	Monitoring passif
MEDIUM	$0{,}5 \leq P < 0{,}8$	Confirmation client
HIGH	$P \geq 0{,}8$	Blocage et investigation

11. Conclusion

Ce projet illustre un résultat qui mérite d'être énoncé explicitement : les algorithmes d'optimisation en machine learning et les méthodes numériques de résolution de systèmes

linéaires ne sont pas deux domaines parallèles, **ils sont le même domaine vu sous deux angles différents**. La descente de gradient converge parce que la log-loss est convexe dans les mêmes conditions que celles qui garantissent la convergence de Gauss-Seidel. Les équations normales existent parce que minimiser une norme quadratique produit un système linéaire. Les valeurs SHAP se calculent en temps linéaire sur notre modèle parce que la linéarité du modèle coïncide avec l'additivité des valeurs de Shapley.

La divergence de Jacobi et la convergence de Gauss-Seidel en 14 itérations ne sont pas des accidents numériques : elles sont des conséquences directes et vérifiables du théorème d'Ostrowski-Reich appliqué à une matrice SDP de conditionnement $\kappa \approx 7$. L'absence de gain de SOR est, de la même façon, prédite par la théorie : la sur-relaxation n'est utile que pour les systèmes mal conditionnés. Ces résultats valident que l'implémentation est correcte et que la compréhension théorique sous-jacente est opérationnelle.

Les perspectives naturelles de ce travail incluent : la comparaison avec des méthodes de factorisation directe (Cholesky, LU) sur la même matrice A ; l'extension à des modèles non linéaires (XGBoost, MLP) pour lesquels l'analyse SHAP apporte une véritable plus-value analytique ; et l'évaluation sur un jeu de données bancaires réel (Kaggle Credit Card Fraud Detection) pour mesurer le transfert des performances.

11. Glossaire des concepts clés

Concept	Rôle dans le projet
Format CSR	Représentation creuse : 100× moins de mémoire que la forme dense
Équations normales	Reformulation ML → système linéaire $Aw = b$ de dimension $p \times p$
$\rho(M_J)$	Rayon spectral de Jacobi : condition de divergence/convergence
Théorème d'Ostrowski	Convergence inconditionnelle de GS pour matrices SDP
$\kappa(A) \approx 7$	Conditionnement qui explique l'absence de gain de SOR
tol $= 10^{-6}$	Critère d'arrêt choisi, pas une limite physique
Sigmoïde $\sigma(z)$	Transformation score linéaire → probabilité
$\nabla_w \mathcal{L}$	Gradient = produit creux $X_b^\top v$, même opération que les solveurs
ROC-AUC	Mesure de discrimination indépendante du seuil τ^*
Class weighting / SMOTE	Deux stratégies de gestion du déséquilibre 95/5
Valeurs de Shapley ϕ_i	Contributions marginales par feature, l'explicabilité réglementaire